

Data Structures - Lab

Topic

Hashing

Objectives

- To understand how **hashing works for fast data access**
- To perform **frequency counting of elements**
- To detect **duplicates efficiently**
- To solve problems like:
 - first repeating element
 - non-repeating element
 - top K frequent elements
- To reduce time complexity from **$O(n^2)$ to $O(n)$** in many problems

Outcomes

1. Implementation of hashing by using unordered map.

Task Instructions

Students are required to complete the following tasks during lab time. Create a private repository on your GitHub accounts. The name of the repository should be **HashingLab**. All source code files must be uploaded to this repository, including the header (.h) and implementation (.cpp) files.

Task 1

Write a function `int repeatelement(int arr[], int n)` ;Find the first repeating element in a given array arr of N integers.

Input:

arr = { -1, 2, -1, 3, 2 }

Ans:-1

Task 2

Given an array of integers and a target value, find two indexes whose values add up to the target.

Use: `unordered_map<int, int>`

Example:

nums = [2, 7, 11, 15]

target = 9

Output: 0 1

Task 3

Given an array, find the k most frequent elements.

Example:

```
nums = [1,1,1,2,2,3]
```

```
k = 2 // first highest repeat, second highest
```

Output: 1 2

Task 4

Given a string, find the first character that appears only once.

Use: `unordered_map<char, int>`

Example:

Input: faiza

Output: f

Task 5

Given an array, display all numbers that appear more than once.

Use: `unordered_map<int, int>`

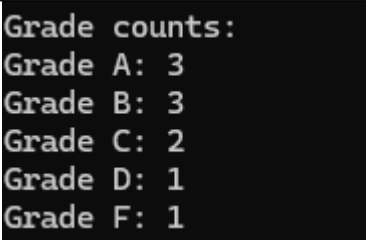
Example:

```
nums = [4,3,2,7,8,2,3,1]
```

Output: 2 3

Task 6

Write a C++ program to count how many students received each grade from A to F from the file using **hashing (unordered map)**

Sample Input	Sample Output
Ahmed A Fatima B Hassan A Layla C Zainab B Bilal D Sara F Kamal A Nadia C Rashid B	 <pre>Grade counts: Grade A: 3 Grade B: 3 Grade C: 2 Grade D: 1 Grade F: 1</pre>

Task 7

Read candidate names from a file where each line represents one vote. Use an `unordered_map` to count votes for each candidate and determine the winner.

Conditions:

- Candidate with maximum votes → Winner
- Others → Loser

Sample Input

```
Ali  
Sara  
Ali  
Ahmed  
Sara  
Ali
```

Sample Output

```
Ali : 3 votes - Winner  
Sara : 2 votes - Loser  
Ahmed : 1 vote - Loser
```